

DOKUMENTACJA PROJEKTU

14. MediaPlayer

Przedmiot: Zaawansowane aplikacje internetowe

Autor	
Rok akademicki	2026/2027
Wersja projektu	1.1

Gorzów Wielkopolski, 2026

Spis treści

1. Wstęp i cel projektu
2. Rozwiązywany problem i zakres funkcjonalny
3. Założenia i ograniczenia
4. Zastosowany stack technologiczny
5. Architektura i struktura katalogów
6. Opis implementacji
7. Format danych i import/eksport
8. Instalacja i uruchomienie
9. Wdrożenie produkcyjne
10. Osadzanie playera na istniejącej stronie
11. Testy
12. Zrzuty ekranu
13. Możliwe rozwinięcia
14. Podsumowanie

1. Wstęp i cel projektu

MediaPlayer jest prostą aplikacją internetową do odtwarzania materiałów wideo. Projekt został przygotowany tak, aby mógł działać jako osobna strona oraz jako komponent dołączany do już istniejącej strony WWW. Głównym elementem jest klasa JavaScript MediaPlayer, która tworzy interfejs odtwarzacza w wybranym elemencie DOM.

Aplikacja nie próbuje odtwarzać pełnej funkcjonalności serwisów takich jak YouTube. Skupia się na wymaganym zakresie: kolejce odtwarzania, sekcjach filmu, komentarzach do sekcji, osi czasu oraz imporcie i eksporcie opisów. Dodatkowo samodzielna strona umożliwia przesłanie pliku wideo na serwer lub dodanie filmu po istniejącym adresie URL. W trybie serwerowym upload może być automatycznie konwertowany przez FFmpeg do MP4 H.264/AAC.

2. Rozwiązywany problem i zakres funkcjonalny

Problemem rozwiązywanym przez projekt jest potrzeba prostego odtwarzania materiału wideo razem z opisem jego struktury. Użytkownik może oznaczyć konkretne fragmenty filmu i dodać do nich komentarze, a następnie przenieść cały opis między przeglądarkami lub instalacjami za pomocą JSON.

2.1. Funkcje użytkownika

- odtwarzanie pliku wideo za pomocą elementu HTML5 <video>,
- dodawanie wielu filmów do kolejki i przechodzenie między nimi,
- automatyczne przejście do następnego filmu po zakończeniu bieżącego,
- usuwanie filmu z kolejki,
- tworzenie sekcji czasowej z nazwą, początkiem i końcem,
- ustawienie początku lub końca sekcji na aktualny czas filmu,
- dodawanie i usuwanie komentarzy przypisanych do sekcji,
- wyświetlanie sekcji na proporcjonalnej osi czasu,
- kliknięcie osi czasu lub sekcji w celu zmiany czasu filmu,
- import i eksport opisu do JSON,
- lokalne zapamiętywanie konfiguracji w localStorage,
- upload filmu na serwer, automatyczna konwersja do MP4 H.264/AAC oraz dodanie filmu po URL lub ścieżce,
- uruchomienie playera jako osobnej strony lub osadzonego komponentu.

3. Założenia i ograniczenia

Projekt ma charakter studencki i celowo pozostaje możliwie prosty. Nie zastosowano bazy danych ani systemu logowania. Opisy playlisty są przechowywane w pamięci przeglądarki lub w pliku JSON, a przesłane filmy jako zwykłe pliki na dysku serwera.

- brak kont użytkowników i uprawnień do poszczególnych playlist,
- brak bazy danych i trwałego serwerowego zapisu sekcji/komentarzy,
- brak streamingu adaptacyjnego HLS/DASH i wyboru wielu jakości; upload jest jedynie normalizowany przez FFmpeg do jednego formatu MP4 H.264/AAC,
- zewnętrzny URL filmu musi być możliwy do odtworzenia przez przeglądarkę,
- walidacja JSON jest podstawowa i sprawdza przede wszystkim obecność playlisty,
- upload ma limit 500 MB i obsługuje zestaw popularnych rozszerzeń, m.in. MP4, AVI, MKV, DivX/Xvid, MOV, WebM, WMV i MPEG,
- interfejs nie zawiera rozbudowanej edycji istniejącej sekcji - można ją usunąć i utworzyć ponownie.

4. Zastosowany stack technologiczny

Warstwa	Technologia	Zastosowanie	Uwagi
Frontend	HTML5	Struktura strony i element <video>	Bez systemu szablonów
Frontend	CSS3	Układ, responsywność, wygląd playera	Osobne style komponentu i strony
Frontend	JavaScript ES6	Logika playera i obsługa interakcji	Bez frameworka frontendowego
Backend	Node.js	Uruchomienie serwera	Skrypt server.js
Backend	Express 4	HTTP i pliki statyczne	Prosty serwer projektu
Backend	Multer 2	Odbiór uploadu plików wideo	Zapis tymczasowy do temp-uploads
Dane	JSON	Import i eksport opisów	Czytelny format tekstowy
Dane	localStorage	Zapamiętanie opisu w przeglądarce	Nie przechowuje samego filmu
Backend / multimedia	FFmpeg	Konwersja formatów i kodeków do MP4 H.264/AAC	Program systemowy uruchamiany przez child_process.spawn()

5. Architektura i struktura katalogów

Projekt ma prosty podział odpowiedzialności. Komponent playera jest oddzielony od kodu strony samodzielnej i od backendu. Dzięki temu ten sam plik media-player.js może zostać dołączony do innej strony bez uruchamiania całej aplikacji demonstracyjnej.

```
MediaPlayer-project/  
|-- server.js  
|-- package.json  
|-- temp-uploads/  
|-- README.md  
|-- public/  
|   |-- index.html  
|   |-- embed-demo.html  
|   |-- assets/  
|       |-- css/player.css  
|       |-- css/app.css  
|       |-- js/media-player.js  
|       |-- js/app.js  
|   |-- data/sample-description.json  
|   |-- media/sample.mp4  
|   |-- uploads/  
|-- docs/
```

5.1. Podział odpowiedzialności

- `media-player.js` - komponent i logika odtwarzania,
- `app.js` - formularze strony standalone, `localStorage` i komunikacja z uploadem,
- `server.js` - serwer HTTP, upload i uruchamianie konwersji FFmpeg,
- `player.css` - style potrzebne samemu komponentowi,
- `app.css` - wygląd dodatkowego panelu strony głównej,
- `embed-demo.html` - przykład użycia komponentu na innej stronie.

6. Opis implementacji

6.1. Klasa MediaPlayer

Najważniejszą częścią projektu jest klasa `MediaPlayer`. Konstruktor przyjmuje element (lub selektor) oraz opcje wejściowe. Następnie tworzy interfejs, podłącza zdarzenia i ładuje dane początkowe.

```
const player = new MediaPlayer('#player', {
  playlist: [
    {
      title: 'Mój film',
      src: '/media/film.mp4',
      sections: []
    }
  ]
});
```

Dane playera są przechowywane w obiekcie `this.data`, natomiast `this.currentIndex` wskazuje aktualnie wybrany film. Widok kolejki, sekcji i osi czasu jest odświeżany metodami renderującymi.

6.2. Kolejka odtwarzania

Metoda `addMedia()` dodaje nowy film do tablicy playlisty. `loadMedia(index)` ustawia adres filmu w elemencie `<video>`. Przyciski Poprzedni i Następny wykorzystują indeks bieżącej pozycji, a zdarzenie `ended` powoduje automatyczne wywołanie `next()`.

6.3. Sekcje i komentarze

Sekcja ma nazwę, czas początku, czas końca i tablicę komentarzy. Przy dodawaniu sekcji sprawdzane jest, czy koniec jest późniejszy niż początek oraz czy nie wykracza poza długość filmu. Sekcje są sortowane po czasie startu.

```
{
  title: 'Omówienie',
  start: 12.5,
  end: 30,
  comments: ['Ważny fragment filmu.']
}
```

6.4. Oś czasu

Po zdarzeniu `loadedmetadata` dostępne jest `video.duration`. Położenie sekcji liczone jest procentowo względem całej długości filmu. Przykładowo sekcja od 20 do 40 sekundy w filmie 100-sekundowym zaczyna się na 20% szerokości osi i ma 20% szerokości.

```
left = section.start / duration * 100%
width = (section.end - section.start) / duration * 100%
```

Marker bieżącego czasu jest przesuwany przy zdarzeniu `timeupdate`. Kliknięcie osi oblicza pozycję kliknięcia względem szerokości i ustawia `video.currentTime`.

6.5. Delegacja zdarzeń i własne zdarzenie zmiany

Przyciski tworzone dynamicznie mają atrybut `data-action`. Zamiast podpinąć osobny listener do każdego z nich, jeden listener w komponencie sprawdza rodzaj akcji. Jest to prosta delegacja zdarzeń.

Po zmianie danych komponent emituje `CustomEvent("mediaplayer:change")`. Strona standalone nasłuchuje tego zdarzenia i zapisuje dane do `localStorage`. Dzięki temu MediaPlayer nie jest na stałe związany z jednym sposobem zapisu.

6.6. Upload plików

Formularz wysyła plik przez `fetch()` jako `FormData` do endpointu `POST /api/upload`. Multer zapisuje oryginał do `temp-uploads`. Następnie Node.js uruchamia FFmpeg przez `child_process.spawn()`. Film jest konwertowany do MP4 z wideo H.264 (`libx264`) i audio AAC, po czym gotowy plik trafia do `public/uploads`, a oryginał tymczasowy jest usuwany. Backend zwraca adres względny gotowego MP4, który jest dodawany do playlisty. Dzięki temu AVI, MKV oraz pliki z kodekami DivX/Xvid nie muszą być odtwarzane bezpośrednio przez przeglądarkę.

Warto rozróżniać kontener i kodek: MP4, MKV i AVI są kontenerami, natomiast H.264, DivX i Xvid są kodekami wideo. Sama końcówka pliku nie gwarantuje zgodności z przeglądarką, dlatego projekt konwertuje upload do jednego przewidywalnego zestawu: MP4 + H.264 + AAC.

```
const response = await fetch('/api/upload', {
  method: 'POST',
  body: formData
});
```

7. Format danych i import/eksport

Eksportowany plik JSON zawiera wersję formatu i playlistę. Każdy film zawiera tytuł, adres źródła oraz sekcje. Każda sekcja przechowuje czas i komentarze. Przy eksporcie dodawana jest też data `exportedAt`, która nie jest konieczna do importu.

```
{
  "version": 1,
  "playlist": [
    {
      "title": "Film demonstracyjny",
      "src": "/media/sample.mp4",
      "sections": [
        {
          "title": "Początek",
          "start": 0,
          "end": 4,
          "comments": ["Przykładowy komentarz"]
        }
      ]
    }
  ]
}
```

Eksport w przeglądarce wykorzystuje `Blob` i tymczasowy URL. Import odczytuje plik metodą `file.text()`, wykonuje `JSON.parse()` i przekazuje dane do `loadDescription()`.

8. Instalacja i uruchomienie

8.1. Wymagania

- Node.js 18 lub nowszy,

- npm,
- FFmpeg dostępny w systemie (polecenie `ffmpeg -version`` powinno działać),
- współczesna przeglądarka internetowa (Chrome, Edge, Firefox itp.).

8.2. Uruchomienie lokalne

1. Rozpakować projekt i przejść do katalogu ``MediaPlayer-project``.
2. Sprawdzić instalację FFmpeg poleceniem `ffmpeg -version``.
3. Zainstalować zależności poleceniem ``npm install``.
4. Uruchomić serwer poleceniem ``npm start``.
5. Otworzyć `http://localhost:3000`` w przeglądarce.
6. Przykład wersji osadzonej znajduje się pod `http://localhost:3000/embed-demo.html``.

```
npm install
npm start
```

Plik ``package.json`` zawiera listę bibliotek wymaganych przez backend. Katalog ``node_modules`` nie powinien być dołączany do archiwum projektu, ponieważ można go odtworzyć przez ``npm install``. FFmpeg jest osobnym programem systemowym i nie znajduje się w ``node_modules``.

9. Wdrożenie produkcyjne

Najprostsze wdrożenie wymaga hostingu obsługującego Node.js oraz zainstalowanego FFmpeg. Po przesłaniu kodu na serwer należy zainstalować zależności i uruchomić ``server.js``. Port może zostać ustawiony zmienną środowiskową `PORT`, a niestandardową ścieżkę do FFmpeg można przekazać przez `FFMPEG_PATH`.

```
npm install --omit=dev
PORT=3000 npm start
```

W typowym hostingu aplikację można wystawić za reverse proxy (np. Nginx), które przekieruje ruch HTTPS do procesu Node.js. Katalog ``public/uploads`` oraz katalog ``temp-uploads`` muszą mieć prawa zapisu dla procesu serwera. W większym systemie filmy powinny być przechowywane w osobnej usłudze plikowej lub object storage, ale w projekcie studenckim zapis na dysku jest wystarczający.

Jeżeli film został już wcześniej wgrany na hosting jako format obsługiwany przez przeglądarkę, nie trzeba korzystać z endpointu uploadu. Wystarczy podać jego bezpośredni adres w formularzu. Dla zewnętrznych źródeł nadal obowiązują ograniczenia kodeków i polityki dostępu po stronie serwera z plikiem. Automatyczna konwersja dotyczy plików przesyłanych przez endpoint uploadu.

10. Osadzanie playera na istniejącej stronie

Do osadzenia komponentu potrzebne są wyłącznie pliki ``player.css`` i ``media-player.js`` oraz element HTML, w którym player zostanie utworzony. ``app.js`` nie jest wymagany, ponieważ obsługuje tylko dodatkowy panel strony samodzielnej.

```
<link rel="stylesheet" href="/assets/css/player.css">
<div id="my-player"></div>
<script src="/assets/js/media-player.js"></script>
<script>
  new MediaPlayer('#my-player', {
    playlist: [
      { title: 'Mój film', src: '/video/film.mp4', sections: [] }
    ]
  });
</script>
```

Pełny przykład znajduje się w `public/embed-demo.html`. W przypadku innej domeny należy zmienić adresy do CSS, JavaScript i filmu.

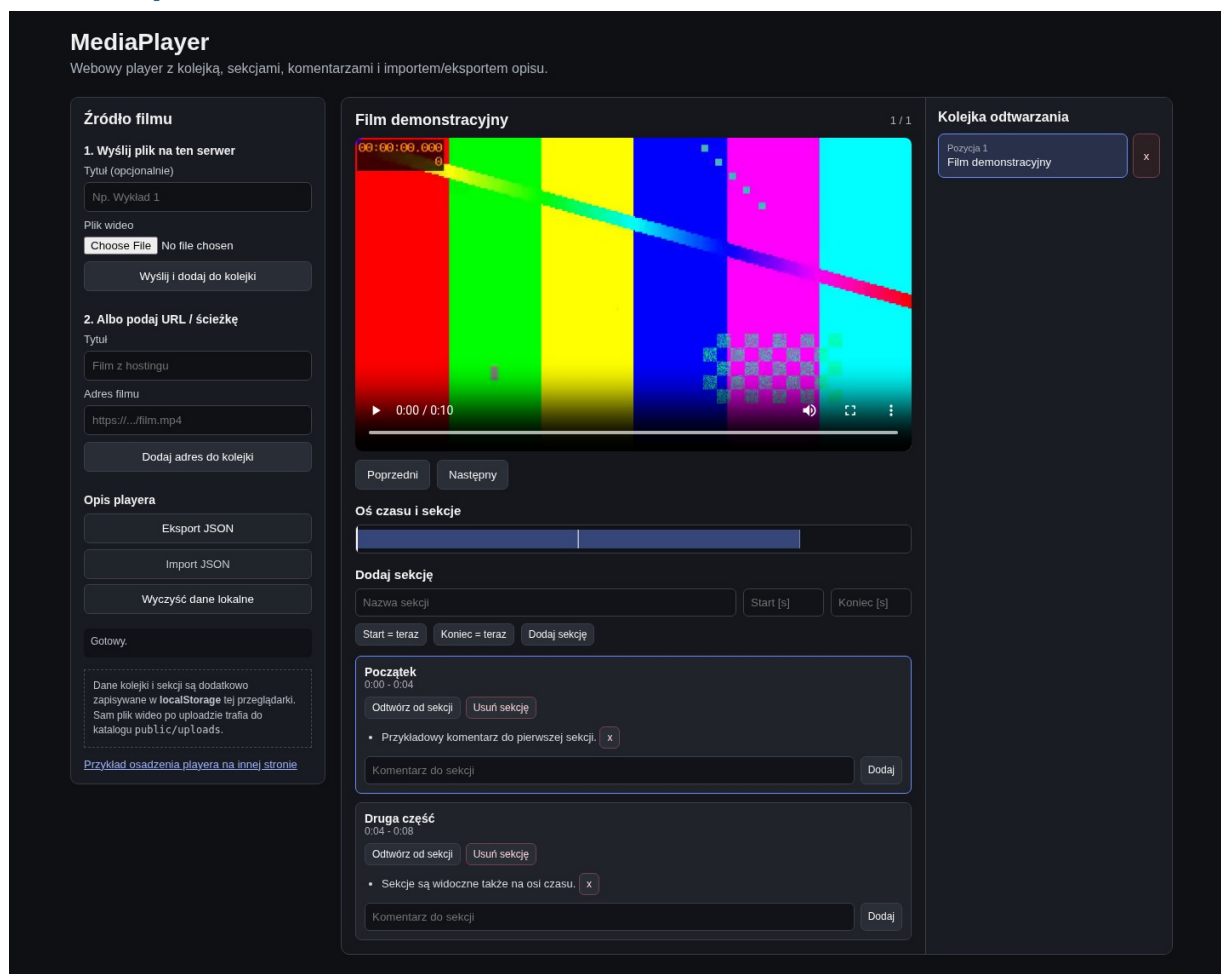
11. Testy

Podczas przygotowania projektu wykonano testy frontendu w przeglądarce oraz sprawdzenie składni plików JavaScript. Szczegółowa lista znajduje się w `docs/TESTY\_MANUALNE.md`.

Scenariusz	Oczekiwany rezultat	Wynik
Start playera	Film demonstracyjny, kolejka i sekcje są widoczne	OK
Metadane filmu	Długość filmu = 10 s, oś czasu działa	OK
Komentarz	Nowy komentarz pojawia się w sekcji	OK
Zmiana filmu	Tytuł i źródło zmieniają się	OK
Usunięcie z kolejki	Pozycja znika i stan jest aktualizowany	OK
Eksport	Dane zawierają version, exportedAt i playlist	OK
Wersja embed	Komponent działa bez app.js	OK
Składnia JS	node --check nie zgłasza błędów	OK
Konwersja AVI/Xvid	Upload tworzy MP4 H.264/AAC w public/uploads	OK
Konwersja MKV	Plik wynikowy MP4 odtwarza się w HTML5 <video>	OK

Test endpointu uploadu należy dodatkowo powtórzyć na docelowym komputerze po `npm install` oraz sprawdzeniu `ffmpeg -version`. Szczególnie warto przetestować AVI/Xvid i MKV, aby potwierdzić, że pliki są konwertowane do MP4 i odtwarzają się w przeglądarce.

12. Zrzuty ekranu



Rys. 1. Samodzielna strona MediaPlayer z kolejką, sekcjami i osią czasu.

Ten dokument udaje zwykłą stronę, do której dołączono komponent MediaPlayer przez CSS i JavaScript.



- baza danych SQLite/PostgreSQL do zapisu playlist, sekcji i komentarzy,
- system kont i uprawnień,
- API REST do zapisu danych po stronie serwera,
- edycja istniejących sekcji i przeciąganie ich granic na osi czasu,
- zmiana kolejności filmów metodą drag & drop,
- miniatury filmów i sekcji,
- napisy, HLS/DASH, kilka jakości odtwarzania i kolejka zadań transkodowania dla dużych filmów,

- przechowywanie plików w zewnętrznej usłudze, np. S3,
- bardziej rozbudowana walidacja importowanego JSON oraz testy automatyczne.

14. Podsumowanie

Projekt realizuje wymagany zakres webowego MediaPlayera. Pozwala pracować z kolejką filmów, sekcjami, komentarzami i osią czasu, a opisy można przenosić za pomocą JSON. Samodzielna strona dodatkowo obsługuje upload, URL filmu oraz konwersję wielu popularnych formatów do MP4 H.264/AAC, natomiast oddzielenie komponentu `MediaPlayer` pozwala na jego użycie na istniejącej stronie.

Zastosowany stack jest nadal niewielki i czytelny: HTML, CSS i JavaScript po stronie klienta oraz Node.js z Express i Multer po stronie serwera. Dodatkowo backend uruchamia FFmpeg do normalizacji plików wideo. Projekt pozostaje celowo nieskomplikowany, dzięki czemu jego kod można prześledzić i wyjaśnić bez dużego frameworka.